

Föreläsning – 260624 – ASP.NET CORE WEB.API – Mer om Identity, JWT och även andra autentiserings-metoder

90% of developers pick the wrong authentication method.

Then spend months patching the consequences.

JWT, Session, OAuth 2.0, API Keys - they're not interchangeable.

Each one solves a different problem.

Here's the breakdown every backend engineer should know 🙌

◆ Session

Server creates a session on login, stores it in Redis/DB, returns session ID via cookie.

→ Stateful. Easy to revoke. Best for traditional web apps with a single backend.

→ Weakness: doesn't scale across services without a shared store.

◆ JWT

Server signs a token with claims (user, expiry, scope).

Client sends it on every request.

→ Stateless. Scales infinitely. Best for microservices, SPAs, mobile apps.

→ Weakness: hard to revoke before expiry, payload is visible by default.

→ JWT är svåra att återkalla före utgångsdatum eftersom de ofta valideras stateless utan server-side token-status.

→ JWT-payloaden är läsbar för den som får tokenen, eftersom den inte är krypterad som standard.

◆ OAuth 2.0

User grants third-party apps limited access via an authorization server.

→ Mixed state. Industry standard for delegation. Best for "Login with Google," GitHub access, third-party integrations.

→ Weakness: complex to implement correctly - auth code, PKCE, client credentials, device flows.

◆ API Keys

Static long-lived secret tied to a service or developer account.

→ Stateful. Simple to rotate. Best for server-to-server APIs, SDKs, internal services.

→ Weakness: no user identity, no expiry by default.
Leaked keys are dangerous.

The rule:

→ Web app, one backend → Session

→ Microservices, mobile, SPA → JWT

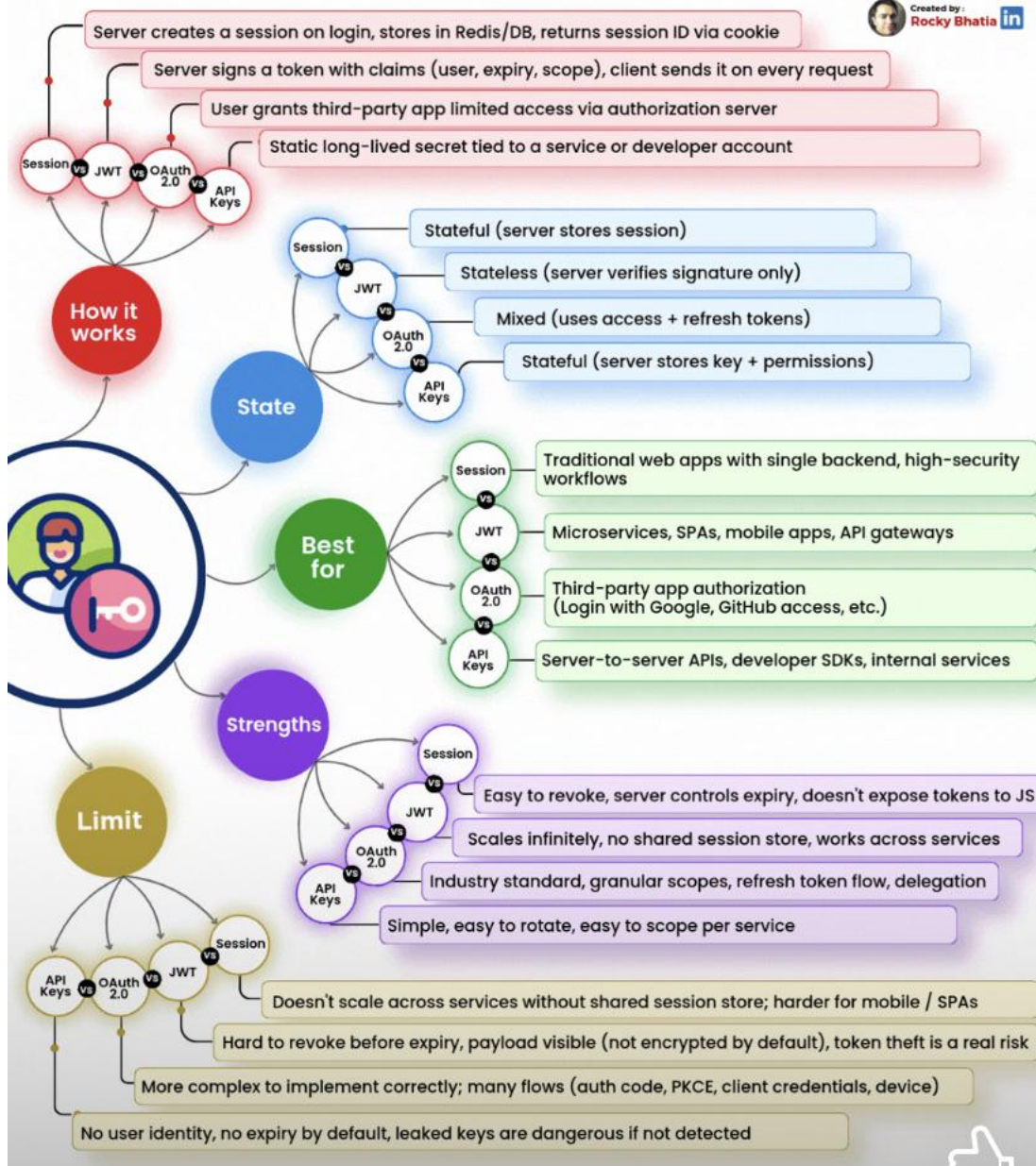
→ Third-party access → OAuth 2.0

→ Server-to-server → API Keys

Picking the wrong one isn't a bug. It's a security incident waiting to happen.

JWT vs Session vs OAuth 2.0 vs API Keys - The Authentication Showdown

Created by:
Rocky Bhatia 



Angående API-Keys:

Server-server: betyder att **en backend pratar direkt med en annan backend**, utan att en människa sitter i en webbläsare emellan.

Ett vanligt exempel är att vårt API hämtar data från en annan tjänst, eller att en extern tjänst skickar order, betalningsinfo eller webhook-events till ditt API. Det är fortfarande en klient-server-modell, men **klienten är en server** i stället för en användare.

Praktiskt:

Om vi har ett internt rapportsystem som anropar <https://api.dittforetag.se/orders>, så kan det systemet skicka en API-key i headern X-API-KEY. Ditt Web API verifierar nyckeln och släpper bara igenom anropet om den är giltig.

Passar bra för:

För interna integrationer, microservices, batchjobb och webhooks.
Om man däremot behöver användar-inloggning, roller eller per-användare-claims är JWT eller annan användarbaserad autentisering oftast bättre.